

SE4095 — Social Network: Theory and Practice

PART 3 OF 3

Community Analysis

Finding, tracking, and evaluating groups in a network — every algorithm explained with when to reach for it.

Covers: what a community is and how it forms · member-based detection (cliques, k-plex, k-core, CPM) · group-based detection (modularity/Louvain, spectral, hierarchical, SBM, InfoMap, label propagation, Girvan-Newman) · community evolution · evaluation with and without ground truth · machine-learning approaches to community detection.

Table of Contents — Part 3

Topic 6 — What Is a Community?

Communities, formation, and why we analyze them

Explicit vs implicit, overlapping vs disjoint

Topic 7 — Community Detection

What is community detection? (Zachary's karate club)

Community detection vs. clustering

Member-based detection: cliques, k-plex, k-core, CPM

Group-based detection: modularity, Louvain, spectral, hierarchical, SBM, InfoMap, label propagation

Girvan–Newman (hierarchical, by edge-betweenness)

Comparing every detection algorithm & picking the right one

Challenges & real-world applications

Topic 8 — Community Evolution

Growth, contraction, merging, splitting, birth, death

Topic 9 — Community Evaluation

Evaluation with ground truth: Precision, Recall, F-measure, Purity, NMI

Evaluation without ground truth

Topic 10 — Machine Learning for Community Detection

Ten ML-based algorithms, consolidated

Exam checklist & quick revision

Topic 6 — What Is a Community?

Social Community

WHAT IS IT?

DEFINITION

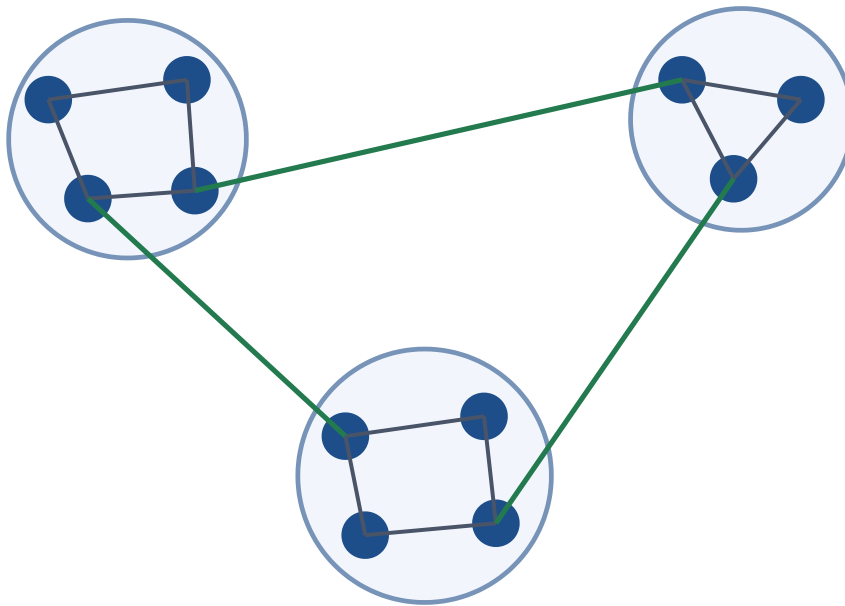
A group of individuals with common economic, social, or political interests or characteristics, often living in relative proximity (the real-world notion) — generalized online to: **a set of at least two nodes sharing some interest, and interactions with respect to that interest.**

WHY DOES IT MATTER — THREE REASONS TO ANALYZE COMMUNITIES

- **Understanding users:** users form groups based on shared interests — the group tells you something no individual profile does.
- **A global view of interactions:** e.g. finding political polarization is a group-level phenomenon, invisible at the individual level.
- **Group-only behaviors:** some Republicans agree with some Democrats individually, but their parties still disagree as blocs — behavior at the group level can differ from what individual-level data would suggest.

Explicit vs Implicit, Overlapping vs Disjoint

Distinction	Meaning	Example
Explicit (emic)	Formed by direct user subscription/membership	Facebook Groups, LinkedIn Groups, Twitter Lists, Google+ Circles
Implicit (etic)	Forms from interaction patterns alone, with no formal subscription	Everyone who frequently calls Canada from the same US area code — a phone company treats them as one marketing community even though no one "joined" anything
Disjoint communities	Every node belongs to exactly one community	Standard partitioning
Overlapping communities	A node can belong to multiple communities at once	A person who is simultaneously part of a family group, a work team, and a hobby club — real social life is usually overlapping



Three implicit communities (dense clusters, enclosed) connected by a few sparse bridge edges (green) — the general shape of "find communities in a graph": dense within, sparse between.

SCENARIO RECOGNITION

If a question describes users being auto-grouped purely from behavior (no sign-up step), that's an **implicit** community — the tools you reach for are detection algorithms (this whole Part), not membership lookups.

Topic 7 — Community Detection

Community Detection

WHAT IS IT?

DEFINITION

The process of finding clusters of nodes ("communities") with **strong internal connections** and **weak connections between different communities**. The ideal is completely disjoint communities with zero inter-community interaction; in practice, algorithms aim to find partitions that are *maximally* decoupled, not perfectly so.

THE CANONICAL EXAMPLE — Zachary's Karate Club

Interactions between 34 members of a real karate club, observed over two years. A real-world disagreement between the club's administrator and its instructor caused the club to split into two factions — members picked a side and one group eventually left to start their own club. When community detection algorithms are run on the interaction data alone (no knowledge of the dispute), **they recover essentially the same two-group split that actually happened**. This is the standard proof-of-concept example cited whenever a course needs to demonstrate that structure alone (without any content/attribute data) can reveal real social fault lines.

Why detecting communities matters

- **Network summarization:** a community can stand in for the whole network — easier to visualize and understand than every individual node.
- **Privacy preservation:** a community can sometimes reveal useful aggregate properties without releasing any individual's private information.

Community Detection vs. Clustering

	Clustering (e.g. k-means)	Community Detection
Data shape	Non-linked — rows in a matrix (a distance/similarity matrix)	Linked — an actual graph
What it operates on	Distance/similarity between feature vectors	The graph structure directly (edges)
Common tools	k-means and other classical clustering	k-clique, quasi-clique, edge-betweenness, modularity, etc.

EXAM TIP — a subtle but favorite trap

If you apply k-means directly to rows of an adjacency matrix, you are only capturing each node's *ego-centric* neighborhood pattern (who it personally connects to), not the deeper graph structure a real community-detection algorithm exploits (multi-hop reachability, shared neighborhoods, global modularity). Don't describe that as genuine community detection in an exam answer — name the distinction explicitly.

Member-Based Community Detection

Group-based algorithms (next section) build communities top-down from the whole graph's structure. Member-based algorithms build bottom-up, starting from small tightly-connected units. The slides organize member-based detection around three "bases":

Basis	Technique	Simple idea
Degree	Clique detection	Members with strong/direct connections to <i>everyone else in the group</i> form a community
Reachability	k-clique, k-club, k-clan	Members that can reach each other within a distance limit form a community
Similarity	Jaccard/Cosine similarity (from Part 2)	Members with similar neighborhoods are grouped together

A. Node Degree — Cliques

WHAT IS IT?

DEFINITION — Clique

A **clique** is a maximum complete subgraph — every node inside it is directly connected to every other node inside it. We look for either the **maximum clique** (the single largest one) or **all maximal cliques** (cliques that can't be extended further, even if smaller than the biggest one).

WHY IS THIS HARD?

Both problems are **NP-hard**. Worse, for a complete graph of just 100 nodes, brute force can generate at least $2^{99}-1$ different cliques starting from a single node — completely impractical.

WORKED EXAMPLE – Pruning to find cliques ≥ 4

Systematic pruning helps: if you're searching for a clique of size k , every member must have degree $\geq k-1$ within the candidate set. To find cliques ≥ 4 , remove all nodes with degree $\leq (4-1)-1 = 2$, then repeat (removing a node can drop a neighbor's degree below the threshold too, so it's applied recursively). On a 9-node graph, this eliminates nodes 9, 2, 1, 3, and 4 one after another, leaving a clean 4-clique {5,6,7,8} behind — far cheaper than brute force.

Why cliques are still a weak tool even with pruning: real cliques are rare, and they're fragile — a clique of 1000 nodes needs $999 \times 1000 / 2$ edges, and removing just *one* of them (under 0.0002% of the total) destroys the clique entirely. This fragility is exactly why relaxed versions (below) exist.

Relaxation	Definition	Relationship to a clique
k-plex	A set of nodes where every member is missing at most $k-1$ connections to the rest of the set	A clique of size k is a special case: a 1-plex
k-core	A maximal connected subgraph where every node has degree $\geq k$ <i>within it</i>	Looser than k-plex — only a minimum-connections guarantee, not near-completeness
k-shell	Nodes that are part of the k-core but NOT part of the $(k+1)$ -core	The "onion layer" exactly between two consecutive cores

EXAM TIP

Finding the maximum k-plex is still NP-hard in general, but the smaller search space makes it far more tractable in practice than finding maximum cliques — a nuanced but examinable point ("NP-hard" isn't the same as "always impractical").

Clique Percolation Method (CPM)

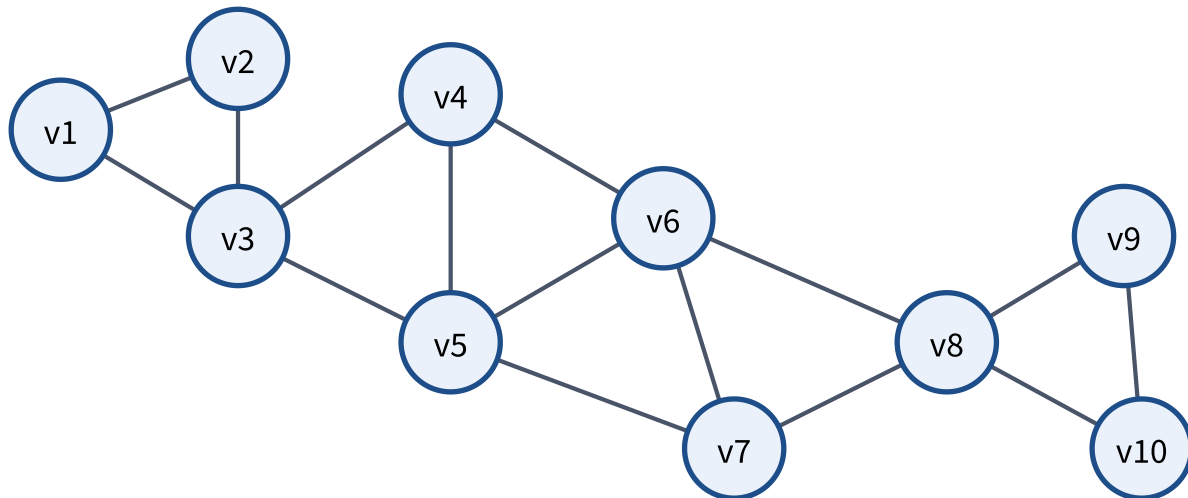
WHAT PROBLEM DOES IT SOLVE?

Cliques are too rigid and rare to use directly as communities — but they're an excellent *seed*. CPM uses small cliques as building blocks and glues overlapping ones together into full (possibly overlapping) communities.

HOW DOES IT WORK?

1. Choose a parameter k and find all cliques of size k in the network.
2. Build a "clique graph": two k -cliques are adjacent if they share $k-1$ nodes.
3. Each connected component of the clique graph becomes one community.

Why this matters: because a single node can belong to multiple overlapping k -cliques, CPM naturally produces **overlapping** communities — unlike almost every other method in this topic.



CPM worked example: size-3 cliques $\{v1, v2, v3\}$, $\{v3, v4, v5\}$, $\{v4, v5, v6\}$, $\{v4, v5, v7\}$, $\{v4, v6, v7\}$, $\{v5, v6, v7\}$, $\{v6, v7, v8\}$, $\{v8, v9, v10\}$. Cliques sharing 2 nodes are chained together into two big overlapping communities, $\{v3, v4, v5, v6, v7, v8\}$ and separately $\{v1, v2, v3\}$ and $\{v8, v9, v10\}$ stay on their own since they only share 1 node with their neighbor clique (not the required $k-1=2$).

Group-Based Community Detection

Group-based methods start from the whole network and look for a global partition, rather than growing outward from small seeds.

Desired community property	Example algorithm/technique
Balanced	Spectral Clustering
Robust	k-edge / k-vertex connectivity
Modular	Modularity Maximization
Dense	Quasi-cliques
Hierarchical	Girvan–Newman Algorithm

Modularity Maximization & the Louvain Method

WHAT IS IT?

DEFINITION — Modularity

Modularity measures the quality of a division of a network into communities: **higher modularity means edges within communities are denser than edges between different communities**, compared against what you'd expect from a random graph with the same degree sequence.

SLIDE GAP — supplementing the standard formula

The slides describe modularity conceptually and use it heavily (Louvain literally optimizes it) but never write out its formula. Since "Modularity Maximization" is a named, examinable method, here is the standard definition used across the field, so you're not caught short:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

m = total edges, k_i = degree of node i , $\delta(c_i, c_j) = 1$ if i and j are in the same community (else 0). The term $k_i k_j / 2m$ is exactly "the number of edges you'd expect between i and j if edges were placed at random" — modularity is literally (actual edges within communities) minus (expected edges within communities if the graph were random).

HOW DOES LOUVAIN WORK?

Louvain is the standard algorithm for maximizing modularity at scale, in two repeating phases:

1. **Phase 1 — Modularity optimization:** start with every node in its own community. For each node, try moving it into each neighbor's community and compute the modularity gain. Move it into whichever community gives the *largest positive* gain; if no move improves modularity, leave it. Repeat until no node wants to move.
2. **Phase 2 — Community aggregation:** build a new, smaller network where each community from Phase 1 becomes a single node. Edges between new nodes are the sum of the original edge weights between the two communities; self-loops represent edges that were already inside a community.

The two phases repeat on the shrinking network, which is exactly why Louvain naturally produces a **hierarchy** of communities and why it typically finishes in fewer than 10 passes — an $O(n \log n)$ algorithm in practice, which is what makes it scale to huge networks.

EXAM TIP — subtleties the slides call out explicitly

Phase 1 is **order-dependent** (the order you visit nodes in affects the exact result and how long it takes), but the final *modularity value* achieved is fairly stable regardless of order — only the computation time really varies. Also: the same node can be reconsidered multiple times within Phase 1, and Phase 1 converges to a *local* optimum, not a guaranteed global one.

Method	How it works	Strength	Real-world example
Spectral Clustering	Uses the eigenvalues/eigenvectors of the graph Laplacian to partition the network	Handles large-scale networks; strong when clusters have a clean mathematical separation	Political networks — identifying factions from collaboration/voting patterns
Hierarchical Clustering	Builds a hierarchy of communities (agglomerative: merge up from individual nodes; divisive: split down from the whole graph), visualized as a dendrogram	Reveals nested community structure at multiple levels simultaneously	Gene-protein interaction networks — general functional groups at one level, specific sub-functions nested inside
Stochastic Block Model (SBM)	Assumes the graph was generated from probabilistic "blocks" — nodes in the same block connect with higher probability; infers the hidden block structure from the observed edges	Principled for networks where ties are probabilistic rather than deterministic	Corporate email networks — inferring departments from who emails whom, without knowing the org chart
InfoMap	Minimizes the description length of a random walk on the network — a walker naturally lingers inside a community before crossing to another	Effective on both small and large, well-defined community structures	Web-browsing behavior — clusters of sites users move between frequently
Label Propagation	Every node starts with a unique label; each node repeatedly adopts the most frequent label among its neighbors until labels stabilize	Very simple, fast, scalable — good for huge, dynamic networks	Twitter interaction networks — communities of users who frequently engage with each other

EXAM TIP — one-line intuitions worth memorizing verbatim

- Label propagation: "look at your neighbors and follow the group that dominates around you."
- InfoMap: "if information/passengers move through this network, where do they tend to remain? Areas with strong internal movement become communities."
- SBM: given only the network, infer the hidden block/community structure purely from connection patterns.

Girvan–Newman Algorithm

SLIDE GAP — supplementing the standard mechanism

The slides name "Girvan–Newman Algorithm" as the example technique for hierarchical community detection but don't walk through its mechanism elsewhere in the deck. Since it's one of the most commonly examined community-detection algorithms in the field (and connects directly to betweenness centrality from Part 2), here is how it actually works:

Girvan–Newman Algorithm

HOW DOES IT WORK?

1. Compute the **edge betweenness** of every edge in the graph (the same betweenness idea from Part 2, but counting shortest paths through an *edge* instead of a node).
2. Remove the edge with the highest betweenness — this is almost always a bridge or local bridge connecting two dense clusters, since inter-community edges sit on many more shortest paths than intra-community edges.
3. Recompute betweenness for all remaining edges (it changes after every removal) and repeat.
4. As edges are progressively removed, the graph splits into more and more components — recording the order of splits produces a full dendrogram, exactly like the hierarchical clustering pattern above.

WHY DOES IT MATTER?

It's the natural "inverse" of Louvain: Louvain *builds up* communities by merging, Girvan–Newman *tears down* the graph by removing the most "between" edges first. It directly reuses betweenness centrality, which is exactly why this algorithm connects two otherwise-separate parts of your syllabus.

EXAM TIP

Girvan–Newman is expensive — recomputing betweenness after every single edge removal is costly on large graphs (recall from Part 2: even a single betweenness computation is $O(nm)$ via Brandes). This computational cost is exactly why Louvain (much cheaper) became the more popular practical choice for large-scale modularity-style detection, even though Girvan–Newman is often taught first conceptually.

Comparing Every Detection Algorithm

The question emphasizes...	Reach for...	Why
"members must all be directly connected to each other"	Clique / k-plex / k-core	These directly encode near-completeness within the group
"communities can overlap, a person belongs to multiple groups"	Clique Percolation Method (CPM)	Almost the only method here that natively produces overlapping output
"maximize internal density vs external sparsity," no other constraint	Modularity Maximization (Louvain)	That's the literal definition of modularity; Louvain is the scalable standard
"nested / multi-level structure" (sub-groups within groups)	Hierarchical Clustering, or Girvan–Newman	Both naturally produce a dendrogram of nested communities
"ties are probabilistic, not certain" (e.g. inferring hidden structure)	Stochastic Block Model	Purpose-built for probabilistic block/community inference
"must scale to a huge, fast-changing network, simplicity matters"	Label Propagation	Cheapest and fastest method on this list, good for dynamic graphs
"clear mathematical/geometric separation between groups"	Spectral Clustering	Explicitly built on the graph Laplacian's eigenstructure

<i>"where does information/flow tend to get 'stuck'"</i>	InfoMap	Directly models a random walker's dwell time
<i>"remove the edges that matter most for connecting distinct groups"</i>	Girvan–Newman	Explicitly targets highest edge-betweenness first

Challenges & Real-World Applications

Four named challenges

- **Scalability:** many algorithms (Girvan–Newman especially) struggle on large-scale networks.
- **Overlapping communities:** genuinely complex to detect nodes belonging to multiple groups correctly.
- **Dynamic networks:** communities change over time (full treatment in Topic 8).
- **Evaluation:** there's no single best way to evaluate detected communities (full treatment in Topic 9).

Domain	Application
Social Networks	Suggesting groups or friendships (e.g. Facebook)
Biological Networks	Grouping proteins and genes by shared function
Marketing	Identifying customer groups for targeted marketing
Fraud Detection	Detecting anomalous communities in financial networks

PRACTICE QUESTIONS — ANSWERED

Q1 (conceptual). Why is finding the maximum clique in a network considered a poor general-purpose strategy for community detection, even though it's a perfectly valid definition of "tightly connected group"?

A: Cliques are NP-hard to find and, more importantly, extremely rare and fragile in real social networks — a large clique requires an enormous number of edges (quadratic in size), and removing even one edge (a tiny fraction of the total) destroys the clique entirely. Real communities are rarely perfectly complete subgraphs, so relaxations (k-plex, k-core) or entirely different approaches (modularity, CPM) are used in practice.

Q2 (scenario). A social platform wants to detect communities but insists that any user can belong to more than one community simultaneously (e.g. a "gaming" group and a "cooking" group). Which class of algorithms should be ruled out immediately, and which specific method directly satisfies this constraint?

A: Rule out standard partitioning methods that assume disjoint output (plain modularity maximization/Louvain, spectral clustering, SBM, label propagation, InfoMap in their basic forms) — these assign each node to exactly one community. The Clique Percolation Method (CPM) directly satisfies the constraint, since it builds communities from overlapping cliques and a node naturally ends up in every community its cliques belong to.

Q3 (conceptual). Explain, in one sentence each, how Girvan–Newman and Louvain are structural opposites of each other despite both producing hierarchical community structure.

A: Girvan–Newman is *divisive* — it starts from the whole connected graph and repeatedly removes the highest-betweenness edges to break it apart into smaller pieces. Louvain is *agglomerative* — it starts from every node as its own community and repeatedly merges nodes/communities together based on modularity gain, building the structure up rather than tearing it down.

Topic 8 — Community Evolution

Community Evolution

WHAT IS IT?

Real networks are dynamic — communities are not static, permanent objects. Studying community evolution means tracking a community across consecutive time snapshots (t and $t+1$) and classifying what happened to it.

Pattern	What happens between t and $t+1$
Growth	New members/nodes join an existing community, expanding it
Contraction	The community loses members, shrinking
Merging	Two (or more) separate communities combine into one
Splitting	One community breaks apart into two or more separate ones
Birth	A brand-new community appears where none existed before
Death	A community dissolves entirely and disappears

SCENARIO RECOGNITION

If a question gives you the *same* network measured at two different points in time and asks what happened to a specific group, map it onto exactly one of these six words — that vocabulary is precisely what's being tested. "A gaming community's members drift into two separate genre-specific subgroups" is **splitting**, not "death" (the community didn't disappear, it fragmented into successors) and not "contraction" (a shrink implies fewer total members, not a division into new communities).

Topic 9 — Community Evaluation

Evaluating Detected Communities

WHAT PROBLEM DOES IT SOLVE?

You ran a detection algorithm and got a partition of the network. Is it actually good? This splits into two completely different situations: do you have a trusted "correct answer" to compare against, or not?

DEFINITION — the perfect community (used to build intuition for every metric below)

Given objects of two different known types (e.g. + and ×), **the perfect community** is one where every object inside it is of the same type. Every metric below is really just a different mathematical way of scoring "how close to perfect is this partition?"

Evaluation With Ground Truth

You have partial or full knowledge of the "correct" community assignments (labels) to check the detected communities against.

Precision, Recall & F-Measure

HOW DOES IT WORK?

DEFINITIONS — the confusion-matrix view

- TP similar members correctly placed in the same community (a correct decision)
- FN similar members incorrectly split into different communities (an incorrect decision)
- FP dissimilar members incorrectly placed in the same community (an incorrect decision)
- TN dissimilar members correctly placed in different communities (a correct decision)

$$Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN}$$

WORKED EXAMPLE – Precision & Recall

Actual \ Predicted	Cluster 1 (Positive)	Cluster 2 (Negative)
Red (Positive)	TP = 5	FN = 2
Blue (Negative)	FP = 1	TN = 6

$$Precision = \frac{5}{5 + 1} = 83.33\% \quad Recall = \frac{5}{5 + 2} = 71.43\%$$

Note: in genuine community-detection evaluation, precision/recall are usually computed by counting *pairs of objects* (are these two nodes correctly kept together or correctly kept apart?) rather than treating this as an ordinary supervised-classification confusion matrix — community detection groups relationships between nodes, it doesn't assign each node an independent class label the way classification does. The pair-counting view is why pair-based evaluation is specifically useful here: a community is fundamentally about which nodes belong *together*, so evaluating pairs directly measures whether the "together vs. separate" structure was preserved.

FORMULA – F-Measure

Precision and Recall each capture only one side of performance. The harmonic mean combines them into one number:

$$F = \frac{2 \times P \times R}{P + R}$$

For the example above: $F = (2 \times 0.6 \times 0.61) / (0.6 + 0.61) \approx 0.60$. (Using pair-counted P and R here, per the slide's worked figure — the exact P/R inputs to F should always be the pair-based versions, not the raw single-object confusion matrix above, if you're asked to compute both consistently.)

Purity

WHAT IS IT?**DEFINITION & FORMULA – Purity**

Assume the majority label inside a community represents that whole community. Purity is the fraction of instances whose label matches their community's majority label.

k number of communities

N total number of nodes

L_j the set of instances with label j across all communities

C_i the set of members in community i

LIMITATION

Purity can be trivially gamed: making every community a singleton (size 1) gives purity = 1.0 (each "community" is 100% one label, since it only has one member) — obviously useless. Very large communities can also distort it. Purity should never be reported alone; pair it with a measure like NMI that accounts for community size/count.

Mutual Information & NMI

WHAT IS IT?

DEFINITION — Mutual Information (MI)

The amount of information two random variables share — how much knowing one variable reduces your uncertainty about the other. Applied here: how much does knowing a node's *detected community* tell you about its *true label*?

FORMULA — Normalized Mutual Information (NMI)

Raw MI is unbounded, which makes it hard to compare across different runs/networks. Normalizing by the entropy of both the labels (H) and the found communities (H) bounds it into a comparable range:

- NMI values close to **1** → high similarity between found communities and true labels.
- NMI values close to **0** → high dissimilarity.

WORKED EXAMPLE — setting up an NMI calculation

14 nodes. Found communities H: [1,1,1,1,1,1,2,2,2,2,2,2,2,2] (6 in community 1, 8 in community 2). Actual labels L: [2,1,1,1,1,1,2,2,2,2,2,2,2,1,1] (7 labeled 1, 7 labeled 2). Cross-tabulating gives the contingency table:

$n_{h,l}$	label=1	label=2	row total (n_h)
community h=1	5	1	6
community h=2	2	6	8
col total (n_l)	7	7	n=14

From here, MI is computed from each cell's $n_{h,l}$ against the row/column totals, then normalized by the entropies $H(H)$ and $H(L)$ to get the final $NMI \in [0,1]$. The community structure here is fairly aligned with the true labels (5/6 and 6/8 are majority-matching within each community), so this would produce a moderately high NMI — a poor/random partition would show cell counts close to what you'd expect from pure chance instead.

Evaluation Without Ground Truth

Most real deployments have no labeled "correct answer" at all — this is the far more common real-world situation.

Approach	How it works
Evaluation with semantics	Analyze posts, profile info, hashtags, and content from each community's members. Extract frequent keywords per community and check whether they form a coherent topic (e.g. "football, match, goal, FIFA, player" → high topic coherence). Human judgment can be crowdsourced at scale (e.g. via Amazon Mechanical Turk).
Clustering quality measures	Run multiple community-detection algorithms on the same network, score each with a quantitative measure, and pick the best-scoring one.

Measure	What it captures
SSE (Sum of Squared Errors)	Cluster compactness — mainly for vector-based clustering, not pure graph clustering
Modularity	Community structure quality (the same Q from Topic 7)
Conductance	How well-separated communities are from each other (low conductance = good separation)
NMI	Agreement with ground-truth communities, <i>when</i> ground truth happens to be available for validation purposes

EXAM TIP

Best practice, stated directly in the slides: **use both semantic coherence and quantitative measures together** for a more reliable evaluation — neither alone is sufficient. This mirrors the Evaluation Dilemma raised all the way back in Part 1's Social Media Mining challenges: "when there's no ground truth, how do you evaluate?" — this whole section is the fuller answer to that exact question.

Exam Checklist — Topics 8 & 9

- Six evolution patterns: growth, contraction, merging, splitting, birth, death — know the distinguishing test between each pair that could be confused (e.g. splitting vs. death; growth vs. birth).
- Ground-truth metrics: Precision/Recall/F-measure (pair-based counting), Purity (majority-label fraction, easily gamed by singletons), NMI (bounded, information-theoretic).
- Without ground truth: semantic coherence (keyword/topic checks) + quantitative measures (SSE, modularity, conductance) — use both together.
- **Common misconception:** purity of 1.0 sounds perfect but is trivially achieved with singleton communities — always sanity-check purity against community count/size, or prefer NMI.

Quick Revision — Topics 8 & 9

Evolution: growth/contraction (size change), merging/splitting (structural combine/divide), birth/death (appear/disappear entirely). Evaluation with ground truth: Precision/Recall/F (pair-based), Purity (majority-label fraction, gameable), NMI (bounded 0–1, information-theoretic, the most robust single number). Without ground truth: semantic coherence (do the keywords make topical sense) plus quantitative measures (SSE/modularity/conductance) — always combine both.

Topic 10 — Machine Learning for Community Detection

Why bring in ML at all?

Every method in Topic 7 works directly on graph structure. Machine-learning approaches instead convert nodes into **feature vectors or embeddings** (numeric representations that capture graph position, content, or both), and then apply standard ML clustering/classification on top. This matters especially when nodes have rich *content* (text, attributes) alongside their connections, or when you want continuous/soft community membership instead of hard partitions.

Algorithm	How it works	Strength	Use case
K-Means	Classical unsupervised clustering — partitions node embeddings into k clusters minimizing within-cluster variance	Simple, efficient, works well with clear separation	Clustering users by activity/profile features
Gaussian Mixture Models (GMM)	Probabilistic clustering — each node gets a probability of belonging to each cluster	Soft clustering — a node can partially belong to multiple communities with different probabilities	Soft clustering by browsing behavior/preferences
DBSCAN	Density-based clustering — groups tightly packed points, marks sparse-region points as noise	No need to specify cluster count in advance; robust to noisy data	Detecting outlier/anomalous researchers in a collaboration network
BERTopic	BERT embeddings + UMAP (dimensionality reduction) + HDBSCAN (clustering) for topic modeling on text	Very strong for text-heavy interaction data	Grouping users discussing similar topics on Twitter/Reddit
LDA (Latent Dirichlet Allocation)	Probabilistic topic-extraction model	Groups nodes by the topics associated with their content	Clustering researchers by the topics of their publications
Node2Vec	Graph embedding via biased random walks over the graph, then standard clustering on the resulting vectors	Preserves both local and global structure in the embedding	Community detection from connection patterns alone
DeepWalk	Graph embedding via random walks treated like sentences (borrowing NLP techniques)	Captures global structure well via long random walks	Large-scale networks with complex global structure (communication, transport)
Graph Convolutional Networks (GCN)	Neural network that aggregates information from each node's neighbors to learn representations	Powerful for semi-supervised detection when <i>some</i> node labels are already known	Citation networks where some papers are already labeled by field
Variational Graph Autoencoders (VGAE)	Encodes graph structure into a latent space, reconstructs the graph from it; latent embeddings are then clustered	Captures non-linear relationships standard methods miss	Complex, non-linear networks (financial, biological)
Hierarchical Dirichlet Process (HDP)	Non-parametric Bayesian hierarchical topic modeling	Finds hierarchical community structure <i>without</i> specifying the number of communities in advance	Hierarchical academic fields/subfields

"WHICH ML APPROACH SHOULD I USE?"

The question emphasizes...	Use...	Why
<i>"a node can partially belong to multiple communities, with a probability"</i>	GMM	The only one here purpose-built for soft/probabilistic membership
<i>"don't know how many communities exist in advance"</i>	DBSCAN or HDP	Neither requires specifying k upfront (HDP also gives you the hierarchy)
<i>"communities defined by what people are talking about," text-heavy</i>	BERTopic or LDA	Both are content/topic-modeling approaches, not pure structural ones
<i>"some ground-truth labels are already available for part of the network"</i>	GCN	The only semi-supervised method on this list
<i>"preserve both who-connects-to-whom AND global structure, feed into standard clustering"</i>	Node2Vec or DeepWalk	Both are graph-embedding methods meant as a pre-processing step before clustering

PRACTICE QUESTIONS — ANSWERED

Q1 (numerical). A detected partition of 10 nodes gives: Community A = {5 nodes, 4 labeled "sports", 1 labeled "news"}, Community B = {5 nodes, 2 labeled "sports", 3 labeled "news"}. Compute the purity of this partition.

A: Purity = (sum of each community's majority-label count) / N = (4 + 3) / 10 = **0.70**. (Community A's majority is "sports" with 4/5; Community B's majority is "news" with 3/5.)

Q2 (scenario). A team wants to evaluate their community-detection output but have no labeled data whatsoever, and their network has no meaningful text/content attached to nodes (just anonymous connection data). Which evaluation approach from Topic 9 is available to them, and which is not?

A: They cannot use ground-truth metrics (Precision/Recall/F/Purity/NMI need labels) and cannot use semantic-coherence evaluation (needs text/content). They're left with purely structural, quantitative clustering-quality measures — modularity and conductance are the two directly computable from connection data alone with no labels or content required.

Q3 (conceptual). A platform wants each user to be able to belong to exactly one primary community but also wants to know the "runner-up" community they almost fit into. Which two of the ten ML methods most naturally provide this, and why?

A: GMM most directly supports this — it gives an explicit probability of membership for *every* community, so the "primary" is the highest-probability one and the "runner-up" is simply the second-highest. HDP could also work if a hierarchical breakdown of memberships is acceptable, but GMM's probabilistic output maps onto "primary + runner-up" far more directly and simply.

Exam Checklist — Topic 10

- ML approaches convert nodes to feature vectors/embeddings first, then apply standard ML — this is the core distinction from Topic 7's direct-on-graph methods.
- GMM = soft/probabilistic clustering (the standout feature to recall).
- DBSCAN and HDP both avoid needing to pre-specify the number of communities (for different reasons — density-based vs. non-parametric Bayesian).
- BERTopic/LDA = content/topic-based, not purely structural.
- Node2Vec/DeepWalk = graph embeddings via random walks, feed into ordinary clustering.
- GCN = the semi-supervised option (needs some existing labels).
- VGAE = captures non-linear structure via a neural autoencoder.

Quick Revision — Topic 10

ML methods embed nodes into vectors (via content, random walks, or neural nets) then cluster. GMM = soft/probabilistic membership. DBSCAN/HDP = no need to pre-specify community count. BERTopic/LDA = topic/content-based. Node2Vec/DeepWalk = structural embeddings via random walks. GCN = semi-supervised (uses some known labels). VGAE = non-linear structure via autoencoding.